

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Malli Chandana	Reg. No.:	192472250
Date:		Duration:	60 minutes

Code of Conduct

I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Malli Chandana

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Fruit Quality Detection Web Application

1. Problem Analysis

1.1 Problem Statement

Fruit quality inspection is an important activity in agriculture, supermarkets, food-processing industries and supply chains. Manual inspection depends heavily on human observation and can be slow, subjective and inconsistent. The proposed Fruit Quality Detection Web Application automates the first level of inspection by accepting a fruit image and classifying it as Fresh or Rotten using a trained Machine Learning model.

1.2 Project Objectives

- Develop an automated fruit-quality classification system.
- Allow a user to upload a fruit image through a web browser.
- Preprocess the uploaded image before prediction.
- Use a trained image-classification model to predict Fresh or Rotten.
- Display the prediction clearly through a Flask web interface.
- Use Git to maintain versions, branches and project history.
- Use Docker to package the application and its dependencies.

1.3 Functional Requirements

- The system shall provide a web page for image upload.
- The system shall accept supported image formats such as JPG and PNG.
- The backend shall validate and preprocess the image.
- The trained model shall generate a quality prediction.
- The result shall be displayed to the user.

1.4 Expected Outcome

The expected outcome is a working web application that provides a quick fruit-quality prediction and can be reproduced on another computer through Git and Docker.

Fruit Quality Detection - System Architecture

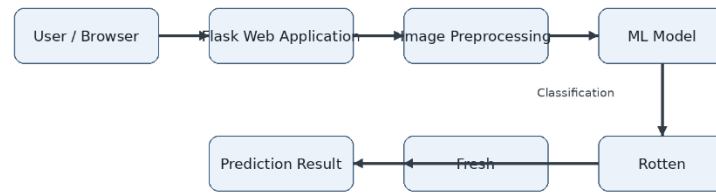


Figure 1.1: Overall Fruit Quality Detection System Architecture

2. Project Structure Design

The repository is organized so that application logic, the trained model, web resources, configuration files and evidence screenshots remain separated. This improves readability and maintainability.

Fruit Quality Detection - Repository Structure

```
Fruit-Quality-Detection/  
├─ app.py  
├─ requirements.txt  
├─ Dockerfile  
├─ docker-compose.yml  
├─ .gitignore  
├─ README.md  
├─ model/  
│   └─ fruit_quality_model.keras  
├─ templates/  
│   └─ index.html  
├─ static/  
│   └─ style.css  
├─ uploads/  
└─ screenshots/
```

Figure 2.1: Proposed Repository Folder Structure

2.1 Major Files and Folders

File / Folder	Purpose	Importance
app.py	Main Flask application and prediction logic	Core backend
model/	Stores trained Machine Learning model	Prediction
templates/	Contains HTML interface	Frontend
static/	Contains CSS and static resources	Presentation
uploads/	Stores uploaded images temporarily	Input handling
requirements.txt	Lists Python dependencies	Reproducibility
Dockerfile	Build instructions for the image	Containerization
docker-compose.yml	Defines Compose service configuration	Deployment
.gitignore	Excludes unnecessary files	Version control
README.md	Project documentation	Maintenance

2.2 Design Justification

The structure separates concerns. The Flask source remains independent from HTML and CSS, the trained model is stored in its own directory, and Docker/Git configuration files are kept at repository level. This makes it easier to modify one part without unnecessarily changing other parts.

3. Git Repository Initialization

Git is a distributed version-control system used to record changes to source code. For this project, Git provides a history of development and allows features to be developed safely through branches.

3.1 Create the Project Folder

```
PS C:\Projects> mkdir Fruit-Quality-Detection
PS C:\Projects> cd Fruit-Quality-Detection
```

3.2 Initialize the Repository

```
PS C:\Projects\Fruit-Quality-Detection> git init
PS C:\Projects\Fruit-Quality-Detection> git status
```

The git init command creates the hidden .git directory. Git status is then used to inspect the current state of tracked and untracked files.

3.3 Stage and Commit

```
git add .
git commit -m "Initial project setup"
```

git add stages files for the next commit. The commit records a permanent snapshot with a meaningful message.

Step 1: Create the Project Folder

```
PowerShell
PS C:\Projects> mkdir hotel-reservation-platform
PS C:\Projects> cd hotel-reservation-platform
PS C:\Projects> dir

Directory: C:\Projects\hotel-reservation-platform

Mode                LastWriteTime         Length Name
----                -
d-----          5/21/2025 10:30 AM             backend
d-----          5/21/2025 10:30 AM             frontend
d-----          5/21/2025 10:30 AM             database
-a-----          5/21/2025 10:30 AM             README.md
-a-----          5/21/2025 10:30 AM             .gitignore

PS C:\Projects\hotel-reservation-platform>
```

What happened?

We created a project folder named `hotel-reservation-platform` and navigated into it. Inside the folder, we created the basic project structure.

Step 2: Initialize Git

```
PowerShell
PS C:\Projects\hotel-reservation-platform> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        backend/
        frontend/
        database/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Projects\hotel-reservation-platform> git add .
PS C:\Projects\hotel-reservation-platform> git commit -m "Initial project setup"
[main (root-commit) a1b2c3d] Initial project setup
5 files changed, 215 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 backend/app.py
create mode 100644 frontend/index.html
create mode 100644 database/schema.sql
PS C:\Projects\hotel-reservation-platform>
```

What happened?

We initialized Git in the project folder.

Added all files and made the first commit with message "Initial project setup".

Step 3: Create Development Branch

```
PowerShell
PS C:\Projects\hotel-reservation-platform> git branch
* main
PS C:\Projects\hotel-reservation-platform> git checkout -b development
Switched to a new branch 'development'
PS C:\Projects\hotel-reservation-platform> git branch
main
* development
PS C:\Projects\hotel-reservation-platform>
```

Explanation:

We created a new branch called `development` to integrate features before merging to main.

Step 4: Create Feature Branch and Add a File

```
PowerShell
PS C:\Projects\hotel-reservation-platform> git checkout -b feature/auth
Switched to a new branch 'feature/auth'
PS C:\Projects\hotel-reservation-platform> echo "# Auth Module" > backend/auth.py
PS C:\Projects\hotel-reservation-platform> git add .
PS C:\Projects\hotel-reservation-platform> git commit -m "Add authentication module"
[feature/auth f1e2d3c] Add authentication module
1 file changed, 1 insertion(+)
create mode 100644 backend/auth.py
PS C:\Projects\hotel-reservation-platform>
```

Explanation:

Created a feature branch `feature/auth`, added a new file for authentication module and committed the changes.

Step 5: Merge Feature Branch into Development

```
PowerShell
PS C:\Projects\hotel-reservation-platform> git checkout development
Switched to branch 'development'
PS C:\Projects\hotel-reservation-platform> git merge feature/auth
Updating a1b2c3d..f1e2d3c
Fast-forward
 backend/auth.py | 1 +
1 file changed, 1 insertion(+)
create mode 100644 backend/auth.py
PS C:\Projects\hotel-reservation-platform>
```

Explanation:

Merged the `feature branch` into `development` successfully.

Step 6: Commit History (git log)

```
PowerShell
PS C:\Projects\hotel-reservation-platform> git log --oneline --graph --all
* f1e2d3c (HEAD -> development, feature/auth) Add authentication module
|
| * a1b2c3d (main) Initial project setup
|/
PS C:\Projects\hotel-reservation-platform>
```

Explanation:

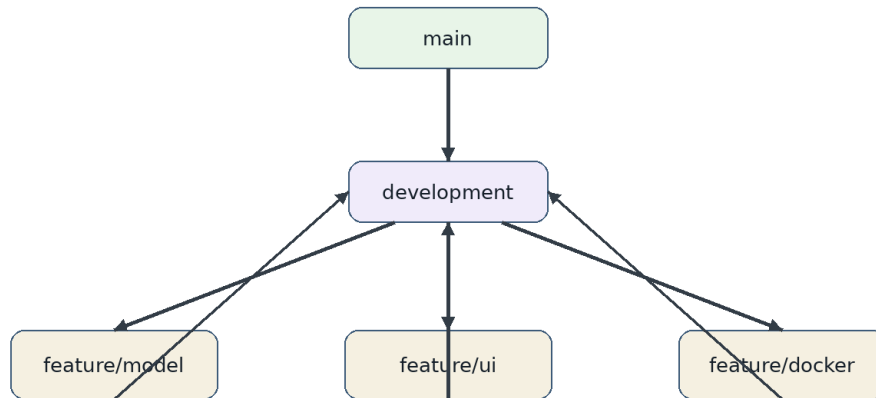
The log shows the commit history with branches and their relationship.

Figure 3.1: Example Git/PowerShell initialization and commit workflow

4. Git Branching Strategy

A simplified GitFlow-inspired model is selected for the project. The stable main branch is protected from unfinished changes. Development integrates tested features, while individual feature branches isolate work.

Git Branching Strategy



Feature branches are merged into development after testing.
Stable releases are merged from development into main.

Figure 4.1: Git Branching Strategy for the Project

4.1 Branches

- main – stable and tested project version.
- development – integration branch for completed features.
- feature/model – changes related to the Machine Learning model.
- feature/ui – changes related to the web interface.
- feature/docker – Dockerfile and Docker Compose changes.
- hotfix – urgent correction to a released version, if required.

4.2 Example Commands

```
git checkout -b development
git checkout -b feature/model
git checkout -b feature/ui
git checkout -b feature/docker
```

4.3 Justification

This strategy is suitable because features can be developed independently and reviewed before integration. It also reduces the risk of breaking the stable main branch and creates a clear history of project development.

5. Version Control Workflow

The Git workflow demonstrates meaningful commits, feature branches, merging and synchronization with a remote repository.

Git + Docker Development Workflow

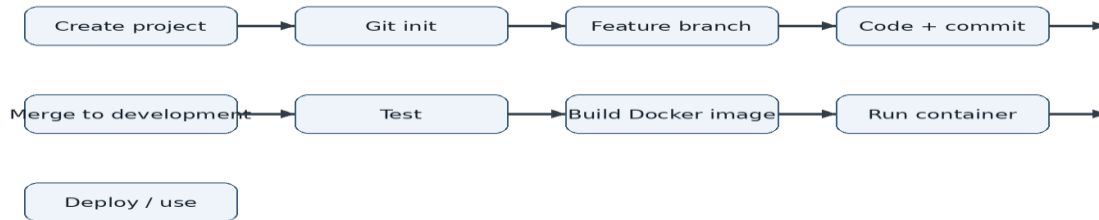


Figure 5.1: Combined Git and Docker Development Workflow

5.1 Feature Development

```
git checkout development
git checkout -b feature/ui
# modify files
git add .
git commit -m "Add fruit image upload interface"
```

5.2 Merge

```
git checkout development
git merge feature/ui
```

After testing, the completed feature is merged into development. Once the complete application is tested, development can be merged into main.

5.3 Remote Synchronization

```
git remote add origin <repository-url>
git push -u origin main
git push origin development
```

5.4 Conflict Resolution

If two branches modify the same lines, Git may report a merge conflict. The developer should inspect the conflict markers, keep the correct content, save the file, stage the resolved file and commit the resolution.

```
git status
git add .
git commit -m "Resolve merge conflict"
```


6. Commit History Analysis

A useful commit history records the project as a sequence of logical changes. Each commit should describe one meaningful change rather than combining unrelated modifications.

6.1 Example Commit History

```
git log --oneline --graph --all
a83f21d Add Docker Compose configuration
7bd34ac Add Dockerfile for Flask application
4ce82fa Add fruit quality prediction functionality
2af61de Add image upload interface
18cd902 Add trained model
93ab781 Add Flask application
51bc320 Initial project setup
```

6.2 Good Commit Messages

- Add Flask application
- Add image upload interface
- Add fruit quality prediction model
- Add Dockerfile
- Add Docker Compose configuration
- Fix image preprocessing error
- Update application documentation

6.3 Why Commit History Matters

Meaningful commit messages help developers understand what changed, locate the source of bugs, compare versions and safely revert unwanted changes. The `--graph` option also helps visualize the relationship between branches.

Evidence to include in the final submission: a screenshot of `git log --oneline --graph --all` and the corresponding GitHub commit history.

7. Docker Environment Design

Docker is suitable because the project depends on Python, Flask, image-processing libraries and a Machine Learning runtime. Manual installation can produce differences between systems. A Docker image packages the application and its dependencies into a reproducible environment.

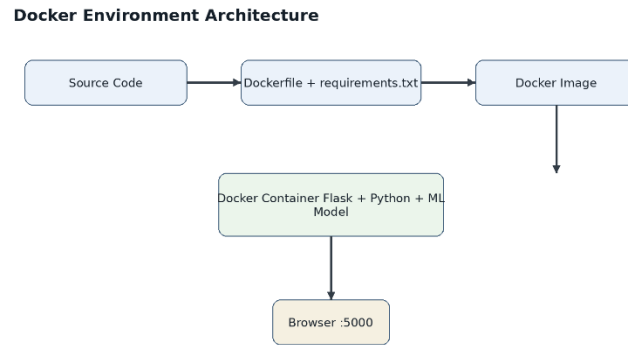


Figure 7.1: Docker Environment Architecture

7.1 Components to Containerize

- Python runtime
- Flask application
- Machine Learning model
- Image-processing and numerical libraries
- Application configuration
- Web interface files

7.2 Benefits for This Project

- Portability across compatible Docker hosts.
- Consistent dependency versions.
- Faster deployment after image creation.
- Isolation from the host Python environment.
- Reproducible application setup.
- Easy integration with automated deployment pipelines.

7.3 Docker Image and Container

A Docker image is the packaged template created from the Dockerfile. A container is a running instance of that image. The application runs inside the container and exposes Flask through port 5000.

8. Dockerfile Development

The Dockerfile describes how the application image is constructed.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

8.1 Instruction Explanation

Instruction	Purpose
FROM	Selects the Python base image.
WORKDIR	Sets /app as the working directory.
COPY requirements.txt .	Copies dependency definitions into the image.
RUN pip install...	Installs required Python packages.
COPY . .	Copies project files into the image.
EXPOSE 5000	Documents the application port.
CMD	Starts the Flask application when the container runs.

8.2 Building the Image

```
docker build -t fruit-quality-app .
```

The command reads the Dockerfile in the current directory, executes its instructions and creates an image named fruit-quality-app.

8.3 Running the Container

```
docker run -p 5000:5000 fruit-quality-app
```

The port mapping makes the Flask application available through the host at port 5000.

9. Docker Compose Design

Docker Compose provides a convenient way to define the application service, ports, volumes and environment configuration in one YAML file.

Docker Compose Architecture

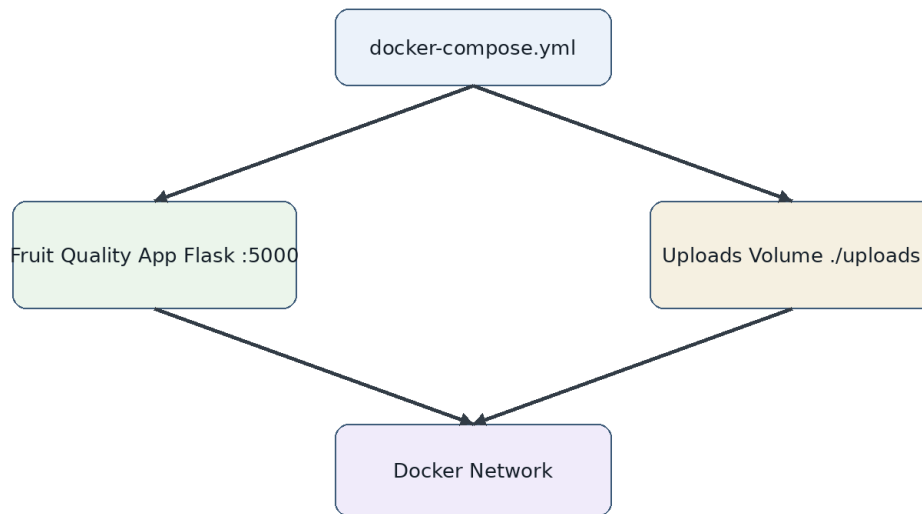


Figure 9.1: Docker Compose Architecture

9.1 docker-compose.yml

```
services:
  fruit-quality-app:
    build: .
    container_name: fruit-quality-app
    ports:
      - "5000:5000"
    volumes:
      - ./uploads:/app/uploads
    environment:
      - FLASK_ENV=production
    restart: unless-stopped
```

9.2 Configuration Explanation

- services defines the application container.
- build: . tells Compose to build from the local Dockerfile.
- ports maps host port 5000 to container port 5000.
- volumes keeps uploaded files accessible through the host directory.
- environment defines runtime configuration.
- restart: unless-stopped improves resilience after unexpected stops.

9.3 Compose Commands

```
docker compose up --build
docker compose up -d --build
```

```
docker compose ps
docker compose logs
docker compose down
```

10. Container Deployment and Testing

10.1 Deployment Procedure

```
docker compose up --build
```

The image is built and the application container is started. The application should then be accessible at <http://localhost:5000>.

10.2 Verify Running Container

```
docker ps
```

The expected output should show the fruit-quality-app container and the port mapping 0.0.0.0:5000->5000/tcp.

10.3 Testing Procedure

- Open the web application in a browser.
- Upload a valid fruit image.
- Confirm that the image is accepted.
- Check that preprocessing and model prediction complete successfully.
- Verify that the result is displayed as Fresh or Rotten.
- Repeat the test with multiple images.
- Inspect Docker logs for application errors.

10.4 Expected Results

```
Fresh fruit image → Prediction: Fresh
Rotten fruit image → Prediction: Rotten
```

10.5 Evidence Screenshots

The final report should include actual screenshots from the student's machine showing docker build, docker compose up --build, docker ps, the browser application and at least one successful prediction. These screenshots provide direct evidence that the containerized system runs successfully.

11. Benefits of Git and Docker

11.1 Git Benefits

- Collaboration: developers can work on separate branches.
- Version management: every important change is recorded.
- Traceability: commit history explains when and why changes occurred.
- Safe experimentation: feature branches isolate unfinished work.
- Recovery: previous versions can be restored when necessary.

11.2 Docker Benefits

- Portability: the same image can be used across compatible environments.
- Consistency: dependencies and runtime configuration are packaged together.
- Deployment: application startup is standardized.
- Isolation: application dependencies are separated from the host.
- Maintainability: the environment can be recreated from the Dockerfile.

11.3 Combined Architecture

Git controls the evolution of the source code, while Docker controls the reproducible runtime environment. Together they create a practical development-to-deployment workflow.

Git + Docker Development Workflow

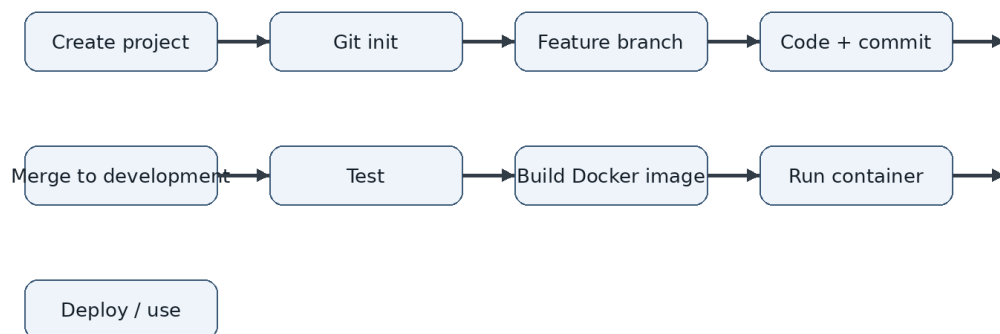


Figure 11.1: Git and Docker Combined Workflow

12. Challenges and Improvements

12.1 Challenges

- Python and Machine Learning library compatibility.
- Large trained model files can increase image size.
- Docker builds may take longer when installing ML dependencies.

- Uploaded images must be validated safely.
- Merge conflicts can occur when multiple branches modify the same files.

12.2 Future Improvements

- Add automated unit and integration tests.
- Introduce CI/CD using GitHub Actions.
- Use a production WSGI server such as Gunicorn.
- Optimize Docker image size with multi-stage builds where practical.
- Add authentication and prediction history.
- Support additional fruit categories.
- Deploy the container to a cloud platform.
- Add monitoring, logging and health checks.

Conclusion

The Fruit Quality Detection Web Application provides a practical scenario for applying both version-control and containerization concepts. The system uses a Flask web application to accept fruit images and a trained Machine Learning model to classify the fruit as Fresh or Rotten.

Git is used to initialize the repository, organize development through branches, record meaningful commits, merge completed features and synchronize the project with a remote repository. The selected branching strategy protects stable code while allowing independent development.

Docker packages the Python runtime, Flask application, Machine Learning model and dependencies into a reproducible environment. Docker Compose simplifies configuration and execution by defining ports, volumes, environment variables and the application service.

The combined use of Git and Docker improves collaboration, version management, portability, deployment and maintainability. Future enhancements such as automated testing, CI/CD, cloud deployment, health monitoring and support for additional fruit classes can make the system more robust.

References / Tools Used

- Git – version control and branching.
- Docker – application containerization.
- Docker Compose – multi-container/application configuration.
- Python – application and Machine Learning programming.
- Flask – web application framework.
- TensorFlow/Keras – Machine Learning model runtime.
- OpenCV/Pillow – image handling where applicable.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25